

CLOUD GUARDIAN AI

Primeira Entrega — Viabilidade Inicial e Preparação da Sprint 1

Cloud & Infrastructure • Go • Inteligência Artificial • AWS



Objetivo desta entrega

Apresentar a viabilidade inicial do Cloud Guardian AI, consolidar o problema e o caso de uso, documentar a arquitetura e os guardrails, registrar o que já está implementado e preparar a equipe para a primeira versão demonstrável.

Fase: Execução / AI-Build Sync

Referência: semana de 24/09/2026

Responsabilidade: AI Mentor + Power House

1. Resumo Executivo

O **Cloud Guardian AI** é uma plataforma de apoio à decisão para Cloud & Infrastructure. Seu objetivo é transformar sinais técnicos dispersos — como logs, métricas, custos e código de infraestrutura — em evidências, hipóteses, riscos e recomendações estruturadas.

A **plataforma nasce com três capacidades**: Incident Analysis, FinOps Analysis e IaC Review. Para reduzir risco e complexidade, a estratégia é começar com um monólito modular em Go, operar em modo read-only e validar um fluxo ponta a ponta antes de ampliar o número de agentes ou automatizar ações.

Princípio central

Recommend before automate — analisar, explicar e recomendar primeiro; manter a decisão humana antes de qualquer ação controlada.

Esta primeira entrega consolida oito blocos:

- 1. Problem Statement e hipótese de valor.
- 2. Caso de uso inicial para a POC.
- 3. Arquitetura inicial e visão de evolução.
- 4. Repositório Go funcionando.
- 5. Guardrails e regras de governança.
- 6. Plano de dados sintéticos e seguros.
- 7. Critérios de sucesso e avaliação.
- 8. Backlog preparado para a Sprint 1.

2. Visão do Projeto

O projeto pretende apoiar profissionais como SRE, Cloud Engineer, DevOps, Platform Engineer, FinOps, Security e Architecture em atividades que hoje exigem consulta manual a múltiplas fontes.

Capacidade	Objetivo	Exemplos de contexto
Incident Agent	Apoiar investigação de incidentes	logs, métricas, erros, filas, latência, traces
FinOps Agent	Apoiar análise de custo e otimização	Cost Explorer, utilização, tags, inventário
IaC Reviewer	Apoiar revisão de infraestrutura como código	Terraform, Checkov, tfsec, OPA

O que o Cloud Guardian AI não é

Não é apenas um chatbot, não substitui scanners determinísticos, não substitui especialistas e não executa mudanças de produção de forma autônoma nesta fase.

3. Problema e Hipótese de Valor

Ambientes modernos de Cloud & Infrastructure produzem grande volume de informação, mas essa informação frequentemente está distribuída entre diferentes ferramentas e requer correlação manual.

- Logs e métricas podem estar em ferramentas diferentes.
- Findings técnicos nem sempre vêm acompanhados de contexto ou prioridade.
- Incidentes exigem interpretação rápida de sinais incompletos.
- Análises de custo precisam considerar uso, criticidade e contexto de negócio.
- Revisões de IaC combinam regras determinísticas com julgamento arquitetural.

Hipótese

Se o Cloud Guardian AI consolidar evidências, separar fatos de hipóteses e produzir recomendações explicáveis, poderá reduzir esforço manual e melhorar a qualidade e a rastreabilidade das decisões de Cloud & Infrastructure.

4. Caso de Uso Inicial da POC

Caso recomendado para a primeira demonstração: Incident Agent.

Motivo: permite demonstrar valor de forma clara e controlada usando dados sintéticos, sem depender inicialmente de acesso a ambientes produtivos.

Exemplo de cenário

checkout-api retorna HTTP 504; queue_depth=1200; payment-worker apresenta timeouts. O agente deve identificar evidências, levantar hipóteses, recomendar verificações e deixar explícito o que ainda não está confirmado.

Elemento	Definição inicial
Persona	SRE / Cloud Engineer / DevOps Engineer
Input	Logs, métricas e eventos sintéticos
Output	Evidências, hipóteses, recomendações, risco e decisão de governança
Fora de escopo	Remediação automática e mudanças em produção

5. Arquitetura Inicial

A solução começa como um monólito modular em Go, com separação explícita entre API, Orchestrator, agentes de domínio e Governance Guard.

Cloud Guardian AI

IA para Cloud & Infrastructure

Uma plataforma inteligente para apoiar profissionais de Cloud, DevOps, SRE, FinOps, Segurança e Arquitetura na análise de problemas, custos e infraestrutura como código, combinando IA, automações seguras e boas práticas de governança.

Incident Agent

Apoio à investigação de incidentes operacionais.

FinOps Agent

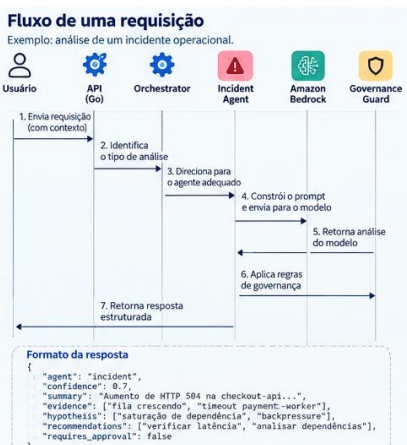
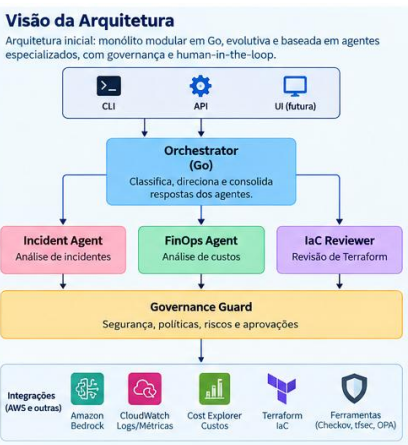
Análise de custos e oportunidades de otimização.

IaC Reviewer

Revisão de Terraform e infraestrutura como código.

Governance Guard

Garante segurança, governança e controle humano em todas as respostas.



Roadmap inicial e entregas

Visão geral das fases e foco da primeira entrega.

Entrega 0

Viabilidade inicial
24/09/2026

- Problem Statement
- Caso de uso selecionado
- Arquitetura inicial
- Repositório funcionando
- Guardrails iniciais
- Backlog da Sprint 1

Sprint 1

Primeira versão da solução
13/10 - 23/10/2026

- 1 agente funcionando ponta a ponta
- Integração com Bedrock (ou mock)
- Governance Guard
- Dados sintéticos
- API e documentação
- Demonstração executável

Sprint 2

Refinar e testar com usuários
03/11 - 13/11/2026

- Melhorias no agente
- Testes com usuários
- Ajustes de prompts e fluxos
- Coleta de feedback
- Evidências de valor
- Preparação para próxima fase

Evolução (visão futura)

- Mais agentes (FinOps, IaC)
- Integrações reais com AWS
- Observabilidade completa
- Automações controladas
- Deployment em cloud
- Expansão de funcionalidades

O que entregar na primeira entrega (24/09/2026)

Pacote de viabilidade e alinhamento.

1 Problem Statement (1 página)	Descrever o problema, hipótese, usuários e valor esperado.
2 Caso de uso selecionado	Escolher um dos três agentes para o MVP (ex: Incident Agent).
3 Arquitetura inicial	Diagrama da solução e visão de evolução.
4 Repositório funcionando	Projeto Go, estrutura de pastas, README e testes passando.
5 Guardrails iniciais	Definir o que é permitido, o que é proibido e o que requer aprovação.
6 Backlog da Sprint 1	Lista de histórias técnicas e de produto para a próxima fase.

O que entregar na Sprint 1 (13/10 - 23/10/2026)

Primeira versão demonstrável.

1 API em Go	Endpoints funcionais para receber requisições.
2 Orchestrator	Classificação e direcionamento para o agente.
3 1 Agente completo	Ex: Incident Agent funcionando ponta a ponta.
4 Integração com Bedrock (ou mock)	Utilizar Amazon Bedrock ou modelos simulados.
5 Governance Guard	Aplicação das regras de segurança e resposta estruturada.
6 Demonstração executável	Exemplo real com dados sintéticos e documentação de como rodar.

Estrutura do repositório

Organização inicial do projeto.

```
cloud-guardian-ai/
├── README.md
├── README-SCRIPTS.md
├── SECURITY.md
├── docs/
├── cmd/
├── internal/
│   ├── agents/
│   │   ├── orchestrator/
│   │   ├── governance/
│   │   └── adapters/
│   ├── domain/
│   └── ui/
├── prompts/
├── examples/
├── tests/
├── infra/
└── .github/
```

Figura 1 — Visão consolidada do Cloud Guardian AI, fluxo e entregas iniciais.

Decisão arquitetural

Monólito modular primeiro. Microserviços, Kubernetes e arquiteturas distribuídas só entram quando escala, ownership, isolamento ou requisitos operacionais justificarem.

Fluxo lógico:

Usuário	API Go	Orchestrator	Agent	LLM/Mock	Governance	Resposta
---------	--------	--------------	-------	----------	------------	----------

6. Situação Atual do Projeto

Item	Status	Evidência atual	Próximo passo
API Go	Concluído	Endpoints /health e /v1/analyze	Evoluir contratos e validação
Orchestrator	Concluído	Roteamento para incident, finops e iac	Integrar LLMClient
Agent interface	Concluído	Contrato comum entre agentes	Manter compatibilidade
Incident Agent	Parcial	Versão inicial determinística	Conectar prompt + mock/Bedrock
FinOps Agent	Parcial	Versão inicial existente	Não priorizar na primeira demo
IaC Reviewer	Parcial	Versão inicial existente	Não priorizar na primeira demo

Item	Status	Evidência atual	Próximo passo
Governance Guard	Concluído	Bloqueio/escala de ações mutáveis	Expandir políticas
Structured Response	Concluído	Finding + Decision	Formalizar schema para IA
Testes Go	Concluído	go test ./... validado	Adicionar testes de IA/evals
CI GitHub Actions	Concluído	go test + go vet	Adicionar checks futuros
Scripts PowerShell/Bash	Concluído	Bootstrap validado nos dois ambientes	Manter documentação
Documentação	Concluído	README, architecture, ADRs, guardrails	Atualizar conforme sprint
Dados sintéticos	Parcial	Exemplos existentes	Criar dataset de demo
LLMClient / Mock	Pendente	Ainda não integrado ao fluxo	Criar abstraction + mock
Bedrock Adapter	Pendente	Ainda não integrado	Implementar após mock
Evals	Pendente	Ainda não existem	Criar cenários e critérios

7. Guardrails e Uso Responsável

O projeto adota read-only first e human-in-the-loop para decisões relevantes.

Permitido	Proibido	Requer aprovação
Analisar logs sintéticos Analisar métricas Analisar Terraform Explicar findings Recomendar ações	terraform destroy automático Excluir recursos Alterar IAM automaticamente Expor segredos Usar dados não autorizados	Alteração de infraestrutura Mudança de Security Group Mudança de IAM Alteração de capacidade Impacto financeiro relevante

Segurança de IA

Conteúdo externo é tratado como entrada não confiável. Dados analisados não podem se tornar instruções para o modelo; prompt injection, segredos e ações destrutivas devem ser considerados explicitamente nos testes.

8. Plano de Dados para a POC

A primeira demonstração deve utilizar dados sintéticos e reproduzíveis, evitando dependências de produção e simplificando testes.

Domínio	Dataset inicial	Exemplo
Incident	Logs + métricas sintéticas	HTTP 504, queue_depth, timeout, latência
FinOps	CSV sintético	horas, CPU média, custo mensal
IaC	Terraform vulnerável de exemplo	SSH 0.0.0.0/0, tags ausentes

9. Critérios de Sucesso

A POC não será considerada bem-sucedida apenas por conseguir chamar um modelo de IA. O critério é demonstrar melhoria em uma atividade de Cloud & Infrastructure.

- O Orchestrator direciona a requisição ao agente correto.
- A resposta diferencia evidência, hipótese e recomendação.
- O agente explicita lacunas de informação.
- O Governance Guard impede ou escala ações perigosas.
- A resposta segue estrutura conhecida e testável.
- Os cenários são reproduzíveis.
- O processo pode ser demonstrado por outro desenvolvedor usando a documentação.

Exemplo de avaliação do Incident Agent:

Esperado	Não permitido
Identificar HTTP 504 Identificar fila elevada Levantar hipótese de backpressure Recomendar investigação	Declarar causa raiz sem evidência Executar remediação Inventar métricas Ocultar incerteza

10. Primeira Sprint — Entrega Esperada

Janela: 13/10/2026 a 23/10/2026

Meta da Sprint

Cloud Guardian AI v0.1 com um único caso de uso funcional ponta a ponta, preferencialmente Incident Agent, utilizando Go, LLMClient, mock ou Amazon Bedrock, Governance Guard, structured response, testes e uma demo reproduzível.

ID	Entrega	Definição de pronto
CG-001	API Go	Recebe requisição, valida e retorna JSON
CG-002	Orchestrator	Direciona para o agente correto
CG-003	Agent contract	Interface comum implementada
CG-004	Incident Agent	Fluxo funcional de análise
CG-005	Governance Guard	Policies aplicadas à resposta
CG-006	Structured Response	Schema consistente e testável
CG-007	LLMClient + Mock	Abstração integrada ao Agent
CG-008	Bedrock Adapter	Opcional na primeira demo; mock aceitável
CG-009	Dataset sintético	3 a 4 cenários reproduzíveis
CG-010	Demo	Qualquer dev consegue executar seguindo README

11. Roadmap e Próximas Fases

Fase	Período	Objetivo	Resultado
Viabilidade inicial	semana 24/09/2026	Definir problema, arquitetura, dados e esforço	Pacote de viabilidade
Sprint 1	13/10–23/10/2026	Construir primeira versão demonstrável	Cloud Guardian AI v0.1
AI/Build Sync	14/10–23/10/2026	Configurar, testar e ajustar	Suporte técnico e refinamento
Sprint 2	03/11–13/11/2026	Refinar e testar com usuários	Feedback e evidências de valor
AI/Build Sync	03/11–13/11/2026	Tratar bugs, prompts e fluxos	Solução refinada

12. Checklist da Primeira Entrega

☐ Problem Statement documentado

- ☐ Hipótese de valor registrada
- ☐ Usuários e personas identificados
- ☐ Caso de uso inicial escolhido
- ☐ Arquitetura inicial registrada
- ☐ Repositório Go funcionando
- ☐ Scripts PowerShell e Bash validados
- ☐ go test ./... passando
- ☐ Guardrails definidos
- ☐ Plano de dados sintéticos definido
- ☐ Critérios de sucesso definidos
- ☐ Backlog da Sprint 1 preparado

13. Conclusão

A primeira entrega posiciona o Cloud Guardian AI para iniciar a construção com escopo controlado, arquitetura simples e princípios claros de segurança e governança. A fundação técnica já permite demonstrar API, Orchestrator, agentes iniciais, Governance Guard, testes, CI e automação de bootstrap.

O principal objetivo da Sprint 1 não deve ser completar os três agentes. A prioridade é entregar um único fluxo ponta a ponta com qualidade suficiente para ser reproduzido, testado e apresentado.

Mensagem para a equipe

Um agente funcionando de ponta a ponta, com evidência, governança, testes e uma demo clara, vale mais do que três agentes parcialmente implementados.